would have to store for each key is large (for instance when keys are long strings) Bloom filters allow to store large sets using only a constant number of bits per key.

  - In order to do so, we have to accept a (small) false positive ratio: sometimes a bloom filter will report it contains an item even if it doesn't – however, no false negatives will be reported.

- **Randomized algorithms**: not all algorithms are deterministic. *Randomized algorithms* are a subset of algorithms whose execution might depend on randomization, and therefore running a randomized algorithm twice on the same input, not always return the same result.
- **Monte Carlo** and **Las Vegas**: randomized algorithms are divided into *Monte Carlo* and *Las Vegas* algorithms, depending on where the uncertainty lies. Monte Carlo algorithms can't guarantee to always return the best possible solution, or a correct response. Las Vegas algorithms, instead, always return best possible solution, but can't offer any guarantees about how long it will take to run.
- **Metrics**: to evaluate the performance of Monte Carlo algorithms, we can use metrics. For binary classification algorithms, which return just `true` or `false` for their input (like the lookup on a Bloom filter), we can use accuracy, precision-recall, of more advanced metrics like *F1*; for Bloom filters we use precision to estimate the ratio of false positives.
- **Tuning**: We can tweak the ratio of false positives for a Bloom filter, and therefore its precision. If we know in